

SRE and DevOps

Gestão de Infraestruturas de Computação

deti universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

João Paulo Barraca

Notes Generated by Gemini Pro and then Reviewed

How Devs differ from Ops

- Development (Dev): Their primary goal is change.
 - They are tasked with writing new code, delivering new features, fixing bugs, and pushing updates to users as quickly as possible.
- Operations (Ops): Their primary goal is stability.
 - They are responsible for keeping the servers running, ensuring security, and maintaining high availability.
 - To Ops, change is the biggest threat to system stability.

GIC

Notice the fundamental clash of incentives. We have our Developers. How are developers traditionally evaluated? By how fast they can ship new features, close Jira tickets, and get their code into the wild. Their entire mandate is change. The business wants agility, and the devs are the engine for that.

But then, we look at Operations. How are system administrators and Ops engineers evaluated? By uptime, performance, and keeping the lights on. They are the ones who get paged at 3:00 AM when a deployment takes down the database. Because most outages are caused by changing a stable system, Ops naturally views change as the enemy. Their mandate is stability.

So, what happens when you have one team paid to push changes, and another team paid to prevent them? You get silos. You get friction.

The classic anti-pattern emerges: Developers finish their code, declare 'it works on my machine,' and metaphorically 'throw it over the wall' to Ops to figure out how to deploy and maintain it in production. When things break, Dev blames Ops for poor infrastructure, and Ops blames Dev for writing bad code.

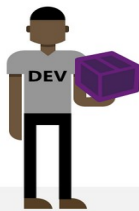
As we transition into the next part of the lecture, keep this conflict in mind. DevOps and SRE are not just toolchains; they are cultural and engineering frameworks designed specifically to tear down this wall and align these competing incentives.

How Devs differ from Ops

- **Dev stereotype**

- Impatient
- Temperamental
- Careless about changes
- Prima donna artists
- Ignorant about the “real world”

“It works on my machine”



- **Ops stereotype**

- Slow to act
- Temperamental
- Too rigid about changes
- Always says “No!”
- Too dumb to understand apps

- “It's not the server, it's your code”



GIC

We can all chuckle at these stereotypes, and if you've done any internships or worked in traditional IT, you've probably heard these exact phrases echoed in the breakroom. But as engineers and future leaders, we need to look past the humor and analyze why these stereotypes exist.

Let's start with the Developers. They are often labeled as impatient prima donnas who are careless about changes. And of course, the most famous battle cry in all of software engineering: 'It works on my machine!' But are developers inherently careless? No. They are reacting to their environment. When they don't have access to production-like environments or lack visibility into how their code actually runs under real-world load, of course they seem ignorant of the 'real world.'

Then we look at Operations. They get labeled as the 'Department of No.' They are seen as rigid, slow-moving BOFHs—Bastard Operators From Hell—who don't understand the application logic and immediately blame the developers' code when things crash. But again, look at their reality: when your job is on the line for every second of downtime, and you are exhausted from being paged at night, extreme rigidity and saying 'No' become your only defense mechanisms.

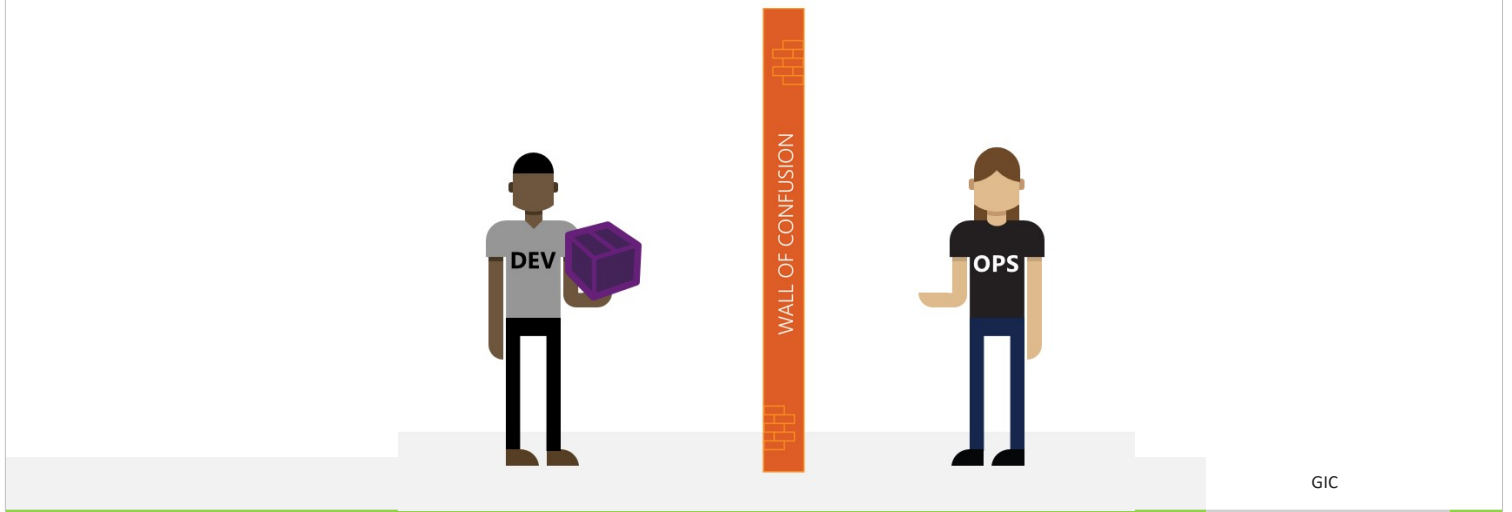
These stereotypes aren't actually character flaws; they are symptoms of a broken organizational system. When teams are siloed, empathy dies. You stop seeing your colleagues as partners and start seeing them as obstacles.

The transition to DevOps and SRE isn't just about learning Kubernetes or CI/CD pipelines. It's about destroying these stereotypes through shared responsibilities, shared tooling, and shared on-call duties. It's about moving from 'It works on my machine' to 'It works in production.'

The Wall of Confusion

A foundational metaphor in DevOps that describes the traditional, siloed barrier between software development teams (Dev) and IT operations teams (Ops).

It represents the inherent conflict of interest, lack of communication, and friction that occurs when these two departments work in isolation.



GIC

We touched on this concept briefly earlier, but here is the famous visualization of what the industry calls the 'Wall of Confusion.' Take a close look at the graphic at the bottom of the slide.

On the left, we have our Dev team holding that purple box. That box represents the finished artifact—the compiled code, the new application, the latest release. In the traditional model, Dev has done their job. They wrote the code, it compiled, and it passed their local tests.

What happens next? Because of this siloed barrier, they metaphorically 'throw it over the wall' to Ops. But notice the problem: there is absolutely no shared context. The Dev team doesn't know exactly how the production servers are configured, and the Ops team has no idea what dependencies or quirks are hiding inside that purple box.

This wall isn't just a physical separation in an office; it is built out of different toolchains, different reporting structures, and a profound lack of communication. When that purple box lands on the Ops side, gets deployed, and inevitably crashes the server, the finger-pointing begins. Dev says, 'You deployed it wrong,' and Ops says, 'You wrote bad code.'

As we dive deeper into DevOps practices, remember this image. Every DevOps practice we discuss—from CI/CD pipelines to infrastructure-as-code—is a direct attempt to tear this specific brick wall down.

The Consequences of the Wall

- **Slow Deployments:** Features take weeks or months to get from development to production.
- **High Failure Rates:** Because Dev doesn't understand the production environment and Ops doesn't fully understand the code, deployments often break.
- **Toxic Culture:** Blame, frustration, and lack of trust build up between the two departments.

GIC

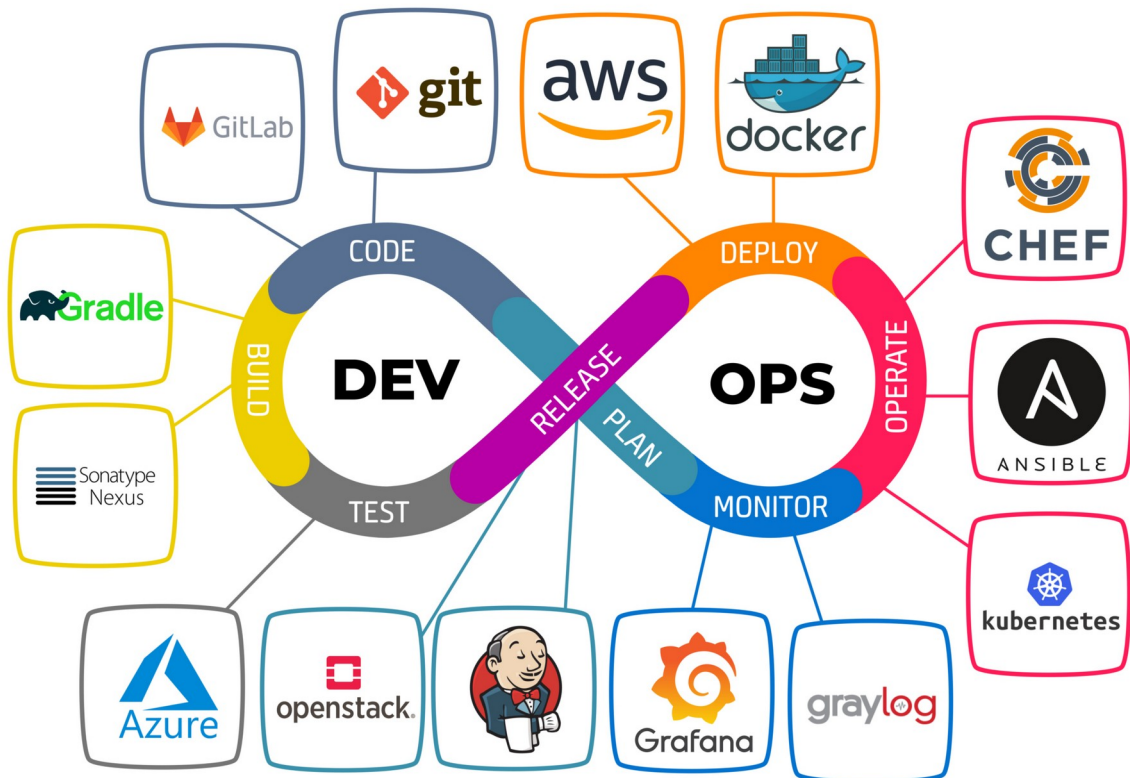
So, we've seen the wall. But what does that wall actually cost a company? It turns out, the consequences are disastrous across the board. Let's break down the three major impacts shown on this slide.

First, Slow Deployments. In the traditional model, releasing software isn't a fluid process; it's a bureaucratic nightmare. When Dev finishes a feature, they don't just push a button. They open a Jira ticket for Ops. Ops puts it in a queue. It might go to a Change Advisory Board (CAB) meeting once a month for approval. The result? Code that took a week to write might take three months to actually reach a customer. In today's market, if you take months to ship a feature, your competitors will eat you alive.

Second, we see High Failure Rates. Why do deployments break so often in this model? Because of environment drift. Dev builds and tests on their laptops or a dev server. But the production environment, managed by Ops, has different security patches, different network rules, and different database configurations. Because they don't share environments or code definitions, the deployment is essentially a blind jump. This is exactly why Friday afternoon deployments used to be a terrifying, forbidden practice in the industry.

Finally, and perhaps most importantly, is the Toxic Culture. This is the human cost. When deployments are slow and constantly breaking, leadership gets angry. When leadership is angry, the blame game starts. Devs get frustrated that their work is stalled; Ops gets burned out from constant firefighting and weekend bridge calls. You lose psychological safety, trust evaporates, and your best engineers will simply quit.

Ultimately, these three consequences, sluggishness, instability, and toxicity are the exact problems that DevOps and SRE were invented to solve.



So, how do we tear down that Wall of Confusion and fix those toxic deployments? We do it by changing the way we work, which brings us to the famous DevOps Infinity Loop.

If you look at the center of this diagram, you'll see the traditional linear process has been bent into a continuous, never-ending cycle. The left side is roughly the Dev domain: Planning, Coding, Building, and Testing. The right side is the Ops domain: Releasing, Deploying, Operating, and Monitoring. But notice there is no wall here. They bleed directly into one another. Most importantly, the end of the Ops cycle—Monitoring—feeds critical data right back into the beginning of the Dev cycle—Planning. That is the feedback loop that makes continuous improvement possible.

Now, let's talk about all these logos surrounding the loop. You'll recognize many of these: Git, Docker, Kubernetes, Jenkins, AWS, Azure. This is the DevOps toolchain.

When Dev and Ops use the same tools and the same automated pipelines, the friction disappears. Dev doesn't 'throw code over a wall'; they commit it to Git, and an automated CI/CD pipeline—maybe using Jenkins or GitLab—builds it, tests it, and deploys it into a container like Docker. The shared tooling creates the shared context that was missing before.

However, I want to give you a very important warning. As engineers, we love tools. It's easy to look at this slide and think, 'If I learn Kubernetes and Jenkins, I am doing DevOps.' That is a dangerous misconception. DevOps is a culture and a methodology. You can buy every tool on this slide, but if your Dev and Ops teams still hate each other and refuse to communicate, you don't have DevOps—you just have very expensive silos.

The tools are the enablers, but the culture is the engine.

What is Site Reliability Engineering

- SRE treats operations as a software engineering problem
 - “Class SRE implements interface DevOps”
- Solves the “Wall of Confusion” Relevant aspects:
 - Isolation/Visibility: limit what can be seen by tenants
 - Resource control: limit resource consumption
 - Portability: reconstruct the same environment in multiple hosts
- Bridges the gap with:
 - Shared Ownership
 - Failure acceptance
 - Capped Operational Work

GIC

Alright, so we've talked extensively about the philosophy of DevOps and the need to break down silos. But how do you actually do that in a concrete, measurable way? This brings us to Site Reliability Engineering, or SRE. SRE was pioneered by Google, and the core definition is right there at the top: SRE is what happens when you ask a software engineer to design an operations team. Instead of managing servers manually, SREs write code to manage systems.

There is a very famous quote in our industry to explain how SRE relates to DevOps: 'Class SRE implements interface DevOps.' For those of you familiar with object-oriented programming, this makes perfect sense. DevOps is the abstract interface—the philosophy that says 'you must collaborate, automate, and share.' SRE is a specific, concrete class that implements those rules with strict metrics and practices.

Now, how does SRE officially dismantle the 'Wall of Confusion'? It uses specific technical strategies:

- Isolation and Visibility: By using containers and proper tenant management, we know exactly how an application is behaving without it bringing down the rest of the server.
- Resource Control: We programmatically define limits (like CPU and memory quotas) so one rogue app can't starve everything else.
- Portability: Through Infrastructure as Code and containerization, SREs ensure that we can reconstruct the exact same environment anywhere. The 'works on my machine' excuse vanishes because the machine environment itself is codified.

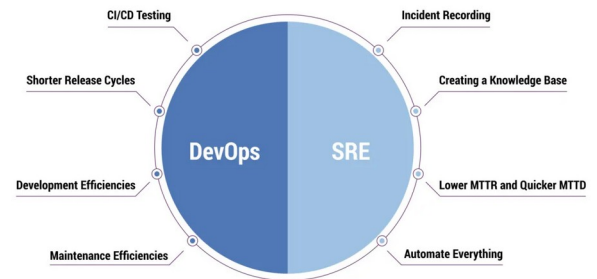
But SRE isn't just technical; it enforces cultural bridges:

- Shared Ownership: Dev and Ops are in the same boat. If a service is too buggy, Devs might have to take the pagers and go on-call until it's fixed.
- Failure Acceptance: This is huge. SRE recognizes that 100% reliability is actually the wrong target. It slows down innovation too much. We use 'Error Budgets' to explicitly allow a certain amount of failure and downtime to keep pushing features.
- Capped Operational Work: SREs strictly limit 'toil'—repetitive, manual sysadmin work—usually to a maximum of 50% of their time. The other 50% must be spent writing software to automate that toil away for the future.

In short, SRE takes the good intentions of DevOps and enforces them with engineering discipline.

Shared Responsibility: Breaking the Silos

- **The Traditional Divide:** Developers want velocity (shipping new features); Operators want stability (keeping systems up).
- **The SRE Shift:** Reliability is no longer just an "Ops problem." It is a shared goal.
- **A Common Language:** Service Level Objectives (SLOs) align both teams on what "healthy" looks like.
- **Unified Ownership:** Developers share the pager; SREs help build the deployment pipelines.



GIC

Good intentions aren't enough. How do we actually force these two historically opposed teams to work together? We do it by entirely redesigning their responsibilities.

Let's look at the first two bullet points. We know the Traditional Divide: Developers are paid for velocity, and Operators are paid for stability. But the SRE Shift changes the paradigm entirely: Reliability is no longer just an 'Ops problem.' It becomes a shared metric that both teams are responsible for. If the system goes down, it's a failure of both the code and the infrastructure.

So, how do we stop the arguments about whether a system is stable enough to handle a new release? We introduce a Common Language, which brings us to the third point: Service Level Objectives, or SLOs. This is one of the most powerful concepts in SRE. Instead of arguing subjectively, Dev and Ops agree on a mathematical target for what 'healthy' looks like—say, 99.9% uptime. If the system is above the SLO, the Developers have the green light to ship new features as fast as they want. If the system drops below the SLO, feature development stops, and everyone—Dev and Ops alike—pivots to fixing reliability. It replaces emotional arguments with data-driven decisions.

Finally, we have Unified Ownership. The days of 'throw it over the wall' are gone. If you look at the diagram on the right, you see DevOps and SRE forming two halves of the same whole. To make this work, SREs help build the CI/CD deployment pipelines on the left side of the circle. They write code to enable developer efficiency. But in exchange, Developers have to share the pain of operations on the right side. Developers now carry the pager. If their code breaks at 3:00 AM, they get woken up alongside the SREs.

When developers feel the operational pain of their own code, the quality of that code miraculously improves.

Failure Acceptance: 100% is the Wrong Target

- **Failure is Inevitable:** Hardware degrades, networks drop, software has bugs, threat actors attack, and humans make mistakes.
- **The Cost of 100%:** Attempting perfect reliability stifles innovation and is cost-prohibitive.
 - 99.9% reliability can cost 10x more than 99%
- **Managing Risk (Error Budgets):** A mathematical allowance for acceptable failure over a period of time.
 - Errors are always present, so there is risk to be minimized everywhere
- **Blameless Culture and Postmortens:** When failures happen, fix the system, don't punish the person.
 - Punishing persons doesn't bring back systems, and will result in toxic culture.

GIC

we arrive at what might be the most counter-intuitive lesson in all of SRE: 100% reliability is the wrong target. > If you tell a traditional executive that your goal isn't 100% uptime, they might look at you like you're crazy. But let's look at the first bullet point. Failure is Inevitable. This is essentially a law of physics in IT. Hard drives fail, cosmic rays flip bits, BGP routes get misconfigured, and humans will eventually make a typo. If you build a system assuming it will never fail, it will collapse catastrophically when it inevitably does.

Secondly, we have to look at the Cost of 100%. There is an exponential cost to adding 'nines' to your reliability. Going from 99% uptime to 99.9% uptime might cost you ten times as much in redundant servers, engineering hours, and testing. But more importantly, it costs you velocity. If you demand perfect stability, developers can never push new features. Attempting 100% reliability actively stifles the business's ability to innovate.

So, if 100% is the wrong target, what is the right one? That brings us to Error Budgets. Instead of demanding zero downtime, we agree on a mathematically acceptable amount of failure. If our Service Level Objective (SLO) is 99.9% uptime, that means we have an 'Error Budget' of 0.1% downtime—roughly 43 minutes a month. That 43 minutes is a currency. We spend that budget by releasing new features, knowing that releases carry risk. If we use up our budget, feature freezes go into effect until the system stabilizes. It perfectly aligns the risk of development with the safety of operations.

Finally, because we accept that failures will happen, we must adopt a Blameless Culture and do Blameless Postmortens. When an engineer accidentally takes down the production database, the traditional instinct is to fire them. But as the slide says, punishing people doesn't bring systems back online. In SRE, we assume good intent. If an engineer made a mistake, it means the system allowed them to make a mistake. We don't fix the human; we fix the system so that the mistake is impossible to make next time. You cannot fire your way to a reliable system.

Capped Operational Work: Taming the "Toil"

- **Toil:** Work that is manual, repetitive, automatable, tactical, and scales linearly with service growth.
- **The 50% Rule:** SREs must spend at least 50% of their time on engineering work, not just operational tasks.
- Software engineering should eliminate manual system administration, allowing auto-scaling
 - Ops are developers of framework tools and automations

GIC

So, if SREs are accepting failure and sharing pagers with developers, what are they actually doing all day? To answer that, we have to define a very specific, almost dirty word in the SRE vocabulary: Toil.

Look at the first bullet point. It is critical to understand that 'toil' does not just mean 'work I don't like doing' or 'administrative overhead.' Toil has a strict engineering definition. It is work that is manual, repetitive, automatable, tactical, and—most dangerously—it scales linearly with service growth.

Think about it this way: if your user base doubles, and you have to manually provision twice as many servers, create twice as many user accounts, and restart twice as many crashed databases, your human effort is scaling linearly with your system. You will eventually run out of humans. That is toil.

To prevent this, SRE introduces The 50% Rule. Traditional operations work behaves like a gas; it will expand to fill all available time if you let it. An engineer could easily spend 100% of their week just closing Jira tickets and fighting fires. In SRE, operational work is strictly capped at 50% of their time.

What happens to the other 50%? That's where the magic happens. Look at the final point on the slide. They spend that time doing actual software engineering to eliminate the manual administration they just dealt with.

If an SRE spends Monday manually scaling up servers to handle a traffic spike, they spend Tuesday writing an auto-scaling script so a human never has to do it again. They aren't just 'sysadmins' closing tickets; they are developers building framework tools, automation platforms, and self-healing systems. They literally engineer themselves out of a job every single day, so they can move on to harder, more strategic problems.

SRE Principles

- **Reliability is King:** SRE considers achieving stability and reliability in a system as the most critical aspect.
- **Users Define Reliability:** SRE considers that monitoring tools do not reflect user experience
 - Components of a user journey must be available when users request it
 - Allows for continuous maintenance tasks
- **Well engineering software/ops will not be enough**
 - Business alignment and effectiveness is required for extra reliability

GIC

Let's distill SRE down to its absolute core principles. If you remember nothing else from today's lecture, remember these three points.

First, Reliability is King. In software engineering, we love to talk about features, UI/UX, and performance. But the hard truth is that reliability is the most important feature of any system. It doesn't matter how beautiful your application is or how advanced your machine learning algorithm is; if the user gets a 504 Gateway Timeout when they try to load the page, your product is completely useless to them. Everything else is secondary to the system actually being there when the user needs it.

That leads directly to the second point, which is a massive paradigm shift: Users Define Reliability. Traditional IT Ops loves their dashboards. They will point to a screen and say, 'Look, all the servers are green, CPU is at 20%, the network is up! The system is healthy.' But if a database lock or a frontend bug prevents a customer from actually checking out their shopping cart, to the customer, the system is down. SRE doesn't care if your servers are green; SRE cares if the user journey is successful.

This is actually a very liberating concept. If we only care that the user journey works, it means we can take individual servers down, reboot databases, and do continuous background maintenance during peak hours—as long as we have the architecture to seamlessly route the user around the maintenance.

Finally, look at the last point. Well-engineered software and ops will not be enough. You can hire the best developers in the world and buy the most expensive cloud infrastructure, but if you don't have business alignment, you will fail. Why? Because achieving 'extra' reliability costs money and slows down feature delivery. If the product managers and executives don't agree with your SLOs and aren't willing to freeze new features when the Error Budget is depleted, your engineering efforts will constantly clash with business demands. Reliability is a business decision, not just an engineering one.

Error Budgets

Critical User Journey (CUJ)

An important workflow that makes the user happy ... or unhappy

SLI

service level **indicator**: a well-defined measure of 'good enough'

SLO

service level **objective**: a top-line target for fraction of good interactions

SLA

service level **agreement**: an SLO with -generally financial- consequences

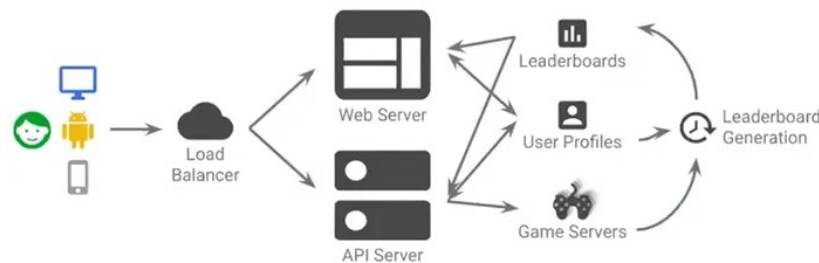


Error Budget

Amount of **unreliability** allowed to operate within SLO

Critical User Journeys

- If users define reliability, user interactions need to be known, understood and measured.
- User Journey happens when users interact with system to reach a goal
- Some are Critical for business



© 2021 Google LLC. All rights reserved.

GIC

Following up on our last point—if we agree that users define reliability, then it is our job as engineers to deeply understand exactly what those users are trying to do. This brings us to the concept of Critical User Journeys, or CUJs.

Let's look at the first two bullet points. A 'User Journey' is simply the path a user takes through our system to achieve a specific goal. But here is the catch: a user journey is rarely just one single API call or one database query.

Take a look at the architecture diagram on the slide. This represents a typical online gaming application. From the user's perspective on the left—whether they are on a phone or a desktop—they are just tapping a button that says 'Play Game' or 'View Leaderboard.' But look at what happens on the backend. That single tap hits a Load Balancer, routes to a Web or API Server, and then fans out to User Profiles, Game Servers, and Leaderboard databases.

If any critical piece of that interconnected web fails or lags, the user fails to reach their goal. And remember, the user doesn't care that the 'User Profile database' is down; they just know the game is broken. Therefore, we have to measure our reliability across this entire path, not just on individual servers.

Finally, look at the last bullet: Some are Critical for business. This is vital for SREs. Not all user journeys are created equal. In this game, if the 'Update Avatar Picture' journey goes down for an hour, it's annoying, but the business survives. However, if the 'Join Game Server' or 'Process Payment' journey goes down, the company is actively losing money and reputation.

As SREs, we identify these Critical User Journeys and build our strictest monitoring, alerting, and Error Budgets around them. We protect the core business first.

Service Level Indicators

- An SLI is a metric created to measure reliability, derived and adapted from the monitoring metrics we use.
- A good SLI, can serve as the basis for determining whether users are happy or unhappy with the service.
 - A CUJ shall have up to 5 SLIs
 - SLIs need to have good signal to noise ratio

$$\text{SLI} : \left(\frac{\text{good events}}{\text{valid events}} \right) \times 100\%$$

Events with non interaction errors or informative responses are not valid (HTTP 4XX, 100).

GIC

So, we've identified our Critical User Journeys. We know what the user is trying to do. Now, how do we actually measure if they are succeeding? We do this using Service Level Indicators, or SLIs.

Look at the first two bullet points. An SLI is a carefully chosen metric that tells us, objectively, whether our users are happy or unhappy. We have hundreds of monitoring metrics available—CPU usage, memory, disk I/O—but remember our rule: users define reliability. A user doesn't care about your CPU usage; they care if the page loaded quickly. Therefore, a good SLI measures the user's experience directly, like the percentage of HTTP requests that returned a successful response in under 200 milliseconds.

Now, how do we calculate this? SRE standardizes this into a very simple, very powerful percentage-based formula

You simply take the number of successful interactions, divide it by the total number of valid interactions, and multiply by 100.

But let's look at the bottom of the slide, because this is where many engineers make a critical mistake. Notice the denominator is 'valid events,' not 'total events.' Why? Look at the HTTP 4XX example. An HTTP 404 means 'Not Found.' If a user accidentally types `yourwebsite.com/dinosaur` and that page doesn't exist, your server will correctly return a 404 error.

Did your system fail? No! It did exactly what it was supposed to do. The user made a mistake. If we included that 404 in our denominator, our reliability score would drop every time a user made a typo. That's why we filter out non-interaction errors or user-caused errors. We only measure the events where the system was actually responsible for the outcome.

Finally, look at the sub-bullets in the middle. A CUJ should have up to 5 SLIs. If you have 20 SLIs for one user journey, you are drowning in data. You lose your signal-to-noise ratio. Pick the 3 to 5 metrics that genuinely represent user happiness—usually focusing on availability, latency, and error rate—and ignore the rest.

Service Level Objectives

- A SLO is an internal objective for operations
 - Should be ambitious
 - Should be achievable
- A SLO should be scoped to the product and user expectations
- SLOs help defining a budget and scoping tasks
 - If the SLO is constantly met, either it is adjusted, or no further optimizations are required

GIC

Alright, so in the last slide, we figured out exactly what to measure using Service Level Indicators (SLIs). But raw data isn't enough. If our SLI tells us our page load success rate is 99.5%, is that good? Is it bad? We don't know until we set a target. That target is the Service Level Objective, or SLO.

Let's look at the first point. An SLO is strictly an internal objective. It's the target your engineering and product teams agree to hit. And setting this number is a delicate balancing act. It must be ambitious enough to keep users happy and competitive in the market, but it must be realistically achievable. Remember our rule from earlier: 100% is the wrong target. If you set your SLO at 100%, you are setting your team up for guaranteed failure.

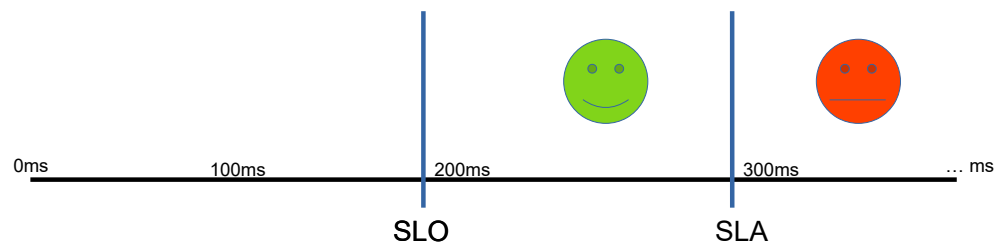
This brings us to the second point: An SLO should be scoped to the product and user expectations. Not all systems need five nines of reliability (99.999%). If you are building an internal time-tracking app for employees, an SLO of 99% might be perfectly fine. But if you are building the payment processing gateway for Amazon on Black Friday, your SLO better be significantly higher. The user's pain tolerance dictates the SLO, not the engineer's ego.

Finally, look at the last two points, because this is where SRE gets really interesting. SLOs help define a budget. Your SLO is the mathematical foundation of your Error Budget. If your SLO is 99.9% uptime, the remaining 0.1% is your budget for downtime, experiments, and feature releases.

And what if you are constantly meeting your SLO? Say your target is 99.0%, but you're consistently hitting 99.9%. A traditional Ops person would celebrate. An SRE looks at that and says, 'We are wasting resources.' If you are over-performing your SLO, it means you are over-engineering for reliability and likely moving too slowly. You should either adjust the SLO to be more ambitious, or stop optimizing the infrastructure and start shipping new features faster until you actually start burning your error budget!

Service Level Agreements

- A SLA is an agreement with stakeholders
 - Define a value above SLOs
 - Violation of SLAs may have financial impact
 - Like SLOs are an agreement with the organization



GIC

We've finally reached the end of the trio. We measured our performance with an SLI, we set an internal engineering target with an SLO, and now we arrive at the Service Level Agreement, or SLA.

Let's look at the first bullet. An SLA is fundamentally different from an SLO because it is a formal agreement with stakeholders, often external customers. This is not just an engineering metric; it is a business contract. And as the second sub-bullet points out, violation of an SLA usually has a financial impact. If you breach your SLA, your company might have to issue account credits, offer refunds, or pay financial penalties to the customer.

Because real money is on the line, we have to look very closely at the relationship between our internal SLO and our external SLA. Take a look at the timeline diagram at the bottom of the slide.

Notice that our internal SLO for latency is set at 200 milliseconds, but our external SLA—the contract we signed with the customer—is set at 300 milliseconds. Why do we 'define a value above SLOs' for the SLA? It's all about creating a buffer.

If you make your SLO and your SLA the exact same number, you are living dangerously. The moment your internal monitoring system pages you to say, 'Hey, we missed our target,' you have already breached the contract and lost the company money.

By keeping the SLA looser than the SLO, you give your engineering team a safety net. If latency creeps up to 250 milliseconds, you miss your internal SLO. The alarms go off, the engineers wake up, and they start fixing the problem. But look at the green face on the diagram—from the customer's contractual perspective, you are still under the 300-millisecond SLA. You fix the issue before the customer has the legal right to complain or demand money.

To summarize the trio: SLIs are what you measure, SLOs are what you want to achieve internally, and SLAs are what you are willing to pay money for if you fail.

Error Budgets

- The Goal: 99.9% uptime for the checkout service over a 30-day window.
 - 30 days = 43,200 minutes.
 - Accepted by all stakeholders
- Error Budget: 0.1% of 43,200 = **43.2 minutes** of allowed downtime.
- The Policy:
 - If Budget > 0: Ship new features!
 - If Budget <= 0: Feature freeze. 100% focus on reliability
 - Error Burndown = when budget goes below 0
- When the window resets, new features can be added.

GIC

Alright, let's do some math. We've talked about accepting failure and setting our SLOs. But how do we use that SLO on a daily basis to make decisions? We use it to calculate our Error Budget.

Let's look at a concrete example. Imagine we have a critical user journey: the 'checkout service' of an e-commerce site. Our stakeholders—engineering, product, and business executives—have all agreed on an SLO of 99.9% uptime over a 30-day window. If we take 30 days and multiply it out (24 hours times 60 minutes), we get 43,200 minutes in our window. Since our goal is 99.9% uptime, that means our allowed failure rate is the remaining 0.1%.

When we calculate 0.1% of 43,200, we get exactly 43.2 minutes. This is our Error Budget. It takes an abstract percentage and turns it into a highly tangible currency. You can look at a developer and say, 'We have 43 minutes of allowed downtime this month.' That is a number humans intuitively understand.

But an Error Budget is completely useless without The Policy to enforce it. This is where SRE puts teeth into the DevOps philosophy. The policy is simple, binary, and non-negotiable:

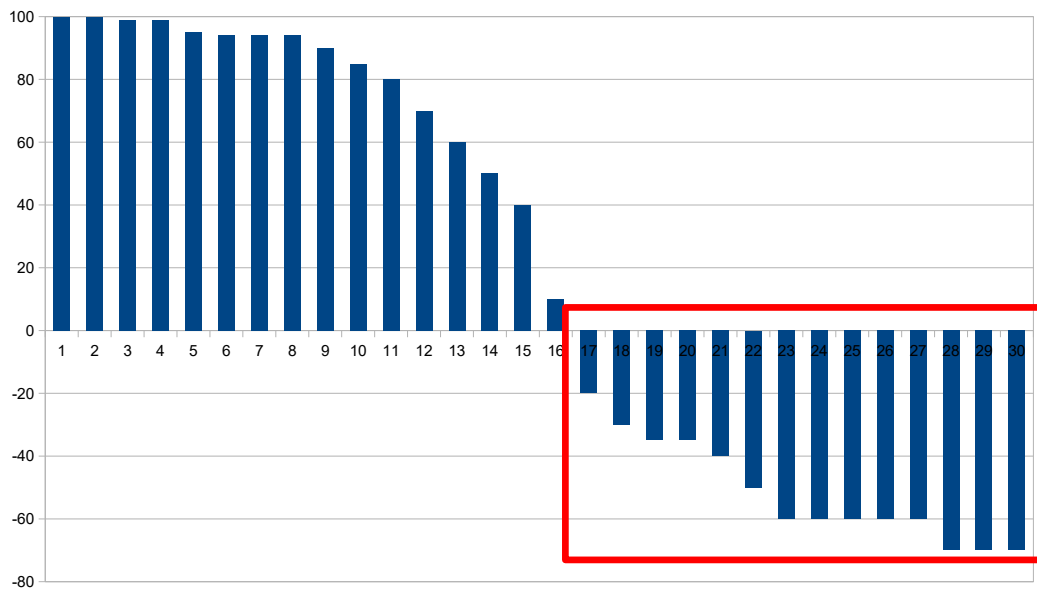
If the Budget > 0: We have minutes left. The system is healthy. Developers have the green light to ship new features, run experiments, and push changes as fast as they want.

If the Budget <= 0: We have burned through our 43 minutes. What happens? Feature freeze. All feature launches are halted. The developers stop writing new product code and pivot 100% to helping Ops fix the reliability issues.

This is called the 'Error Burndown.' And remember, because the stakeholders agreed to the SLO up front, the product managers cannot complain when the feature freeze happens. The math makes the decision, not office politics.

Finally, look at the last point. This punishment isn't forever. It's a 30-day window. When the window resets, we get our 43 minutes back, the freeze lifts, and innovation resumes. This elegant system perfectly balances the Dev's desire for speed with the Ops' need for stability.

Error Budgets (EB) Burndown



GIC

Let's take the math we just discussed and look at how it actually plays out in the real world over a 30-day window. This is a classic Error Budget Burndown chart.

On the X-axis, we have the days of the month, from 1 to 30. On the Y-axis, we have our remaining Error Budget, starting at 100% of our allowance at the top, and dipping into the negatives at the bottom.

Let's walk through the story this graph is telling us:

Days 1 through 10: Things are going pretty well. We are doing normal deployments, maybe we have a few minor hiccups or tiny latency spikes. We are burning a little bit of our budget, but the slope is gentle. This is normal, healthy operation.

Days 11 through 15: Something goes wrong. Maybe a new feature introduced a memory leak, or a database migration went poorly. Notice how steep the drop-off is here. We are hemorrhaging our allowed downtime.

Day 16 into 17: We hit the zero line. We have entirely consumed our 43 minutes—or whatever our budget was—for the month.

Now, look at that massive red box encompassing the second half of the month. This is the danger zone. Our budget is negative.

Remember our policy from the last slide? What happens in this red box? All feature development stops. The CI/CD pipelines are locked for new product code. If a product manager comes to the Dev team on Day 20 and says, 'I need this shiny new button deployed right now,' the answer is an objective, mathematically backed 'No.' >

From Day 17 to Day 30, the entire engineering organization shifts its focus. Instead of building new things, they are fixing the memory leak, adding better test coverage, and improving database resilience so this doesn't happen again.

Finally, when we cross from Day 30 back to Day 1 of the next month, the window resets. We get our budget back, the red box disappears, and the developers are unblocked to start shipping features again.

Risk Estimation

- **ETTD** - Estimated Time To Detection – how long it would take to detect and notify a human (or robot) that the incident has occurred;
 - Also **MTTD** (Mean Time to Detect)
- **ETTR** - Estimated Time To Resolution – how long it would take to fix the incident once the human (or robot) has been notified;
 - Also **MTTR** (Mean Time to Repair).
- **ETTF** - Estimated Time To Failure – estimated frequency between instances of this incident;
 - Also **MTBF** (Mean Time Between Failure).
- **% of Users Affected** – Percentage of users was affected by the failure

GIC

So, we just saw the terrifying red box of the Error Budget burndown. If we want to avoid that red box, we have to deeply understand how incidents consume our budget. To do that, SREs use a specific set of metrics to estimate and measure risk. You will hear these acronyms constantly in the industry.

You'll notice the slide uses 'E' for Estimated and 'M' for Mean. In practice, 'Estimated' is used when we are planning architecture and doing risk forecasting, while 'Mean' is used to average our historical data. Let's break down the incident lifecycle:

First is ETTD or MTTD (Time to Detect). This is the ticking clock from the exact second a system breaks to the moment a human or an automated system actually realizes it's broken. Silent failures are the absolute worst enemy of an SRE. If a payment gateway goes down, but your monitoring doesn't catch it for three hours, you just burned three hours of your Error Budget before you even started fixing it! Lowering your MTTD is all about building better, more sensitive alerts.

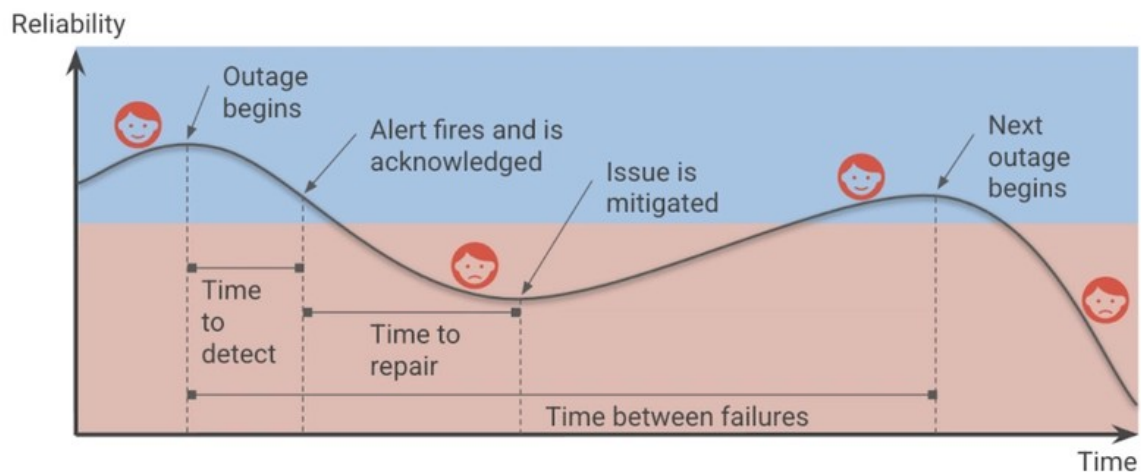
Next is ETTR or MTTR (Time to Resolution/Repair). The alarm is ringing, the engineer is awake. How long does it take to stop the bleeding? A high MTTR usually means poor documentation, complex code, or slow deployment pipelines. A low MTTR usually means the team has great playbooks or uses automated rollbacks—if a bad deployment goes out, the system automatically reverts it in two minutes.

Third, we have ETTF or MTBF (Time to Failure). This measures frequency. Are we dealing with a highly unstable system that crashes every two days, or a rock-solid system that has a weird memory leak once every six months?

Finally, we must measure the % of Users Affected, often called the 'Blast Radius.' An incident that takes down the entire site for 100% of users burns your budget much faster than a localized bug that only affects 2% of users on a specific legacy browser.

If you combine these concepts, you get the formula for your total risk: The frequency of the failure, multiplied by the total time it takes to detect and fix it, multiplied by the blast radius. Our job as SREs is to shrink every single one of those variables.

Risk Estimation



Google

GIC

Let's take all those acronyms we just defined—MTTD, MTTR, MTBF—and put them on a timeline. This graph perfectly illustrates the anatomy of a real-world outage.

On the Y-axis, we have Reliability. Notice the two zones: the blue zone at the top where our users are happy, and the red zone at the bottom where our users are unhappy because we are dropping below our SLO.

Let's trace the line from left to right. Everything is fine, the user has a happy face, and then... the Outage begins. A database locks up, or a bad code deployment starts dropping requests. Notice that the reliability line starts dropping immediately, but look at the gap between 'Outage begins' and 'Alert fires.'

This gap is our Time to Detect (MTTD). This is the most dangerous part of the graph. Why? Because the system is degrading, users are starting to fail their journeys, and we don't even know it yet. If your MTTD is long, your users will be the ones notifying you of the outage via angry posts on Twitter, rather than your own monitoring tools.

Once the alert fires and an engineer acknowledges it, we enter the Time to Repair (MTTR) phase. The system hits rock bottom—look at the sad user face—as the engineer diagnoses the issue, writes a fix, or rolls back the deployment. Eventually, the issue is mitigated, and reliability climbs back into the blue zone. The user is happy again.

Finally, the distance from the start of this outage to the start of the next one is our Time Between Failures (MTBF).

If you think back to our Error Budget calculation, the physical space between the reliability line and that middle threshold—the area dipping into the red zone—represents the actual Error Budget we just burned. To protect our budget, we have to shrink the 'Time to Detect' and 'Time to Repair' windows as much as humanly possible.

Risk Estimation

- For every identified risk, ETDD, ETTR, %impact and ETTF is observed
- Data from actual incidents can be used to define probability

	(in minutes)			(in days)			
Risk	ETTD	ETTR	% Impact	ETTF	incidents/year	bad mins/year	
Disaster recovery drill	5	60	100%	365	1.00	65	
Update major code version	5	120	5%	1095	0.33	2	
Code deployment misconfiguration	30	180	40%	607	0.60	51	
Aggressive database query causing system to down	30	30	30%	60	6.09	110	
Software defects in the code	60	240	20%	219	1.67	100	
Breakdown of caching service	30	240	30%	365	1.00	81	
Outage in the availability zone of the data center	5	60	33%	1095	0.33	7	
Unintended data loss or corruption	240	1,440	5%	1095	0.33	28	
Malicious security breach/attack	80	1,440	50%	487	0.75	570	
High volume of traffic	240	60	25%	1095	0.33	25	
Breakdown in the message queue system	5	300	25%	548	0.67	51	
Disk failure	30	60	1%	1460	0.25	0	
Third-party dependencies failure	240	900	20%	548	0.67	152	

<https://dev.to/coingecko/error-budgets-in-practice-a-data-driven-approach-to-risk-and-release-management-2m9b>

GIC

So, we understand the timeline of a single incident. Now, let's zoom out. How do we manage the risk of all possible incidents across a massive system? We build a risk matrix, just like the one on this slide.

Look at the first column. To protect our Error Budget proactively, we list out every single thing that could go wrong. Bad code deployments, database query timeouts, data center outages, even scheduled disaster recovery drills.

Then, for each risk, we use our historical data—our past postmortems and incident reports—to estimate the variables we just learned: ETDD, ETTR, Impact, and Frequency.

But the real magic of this spreadsheet is the final column on the far right: bad mins/year. This tells us exactly how much of our precious Error Budget we expect this specific issue to consume over the next 12 months.

Let's look at the math hiding in this table. How do we calculate those bad minutes? The formula is brilliantly logical. You take the total time an incident lasts—which is the Time to Detect plus the Time to Resolve (ETDD + ETTR). You multiply that by the blast radius (% Impact). Then, you multiply that by how often it happens (incidents/year).

Let's test it on the 'Software defects in the code' row.

ETDD is 60 mins + ETTR is 240 mins = 300 minutes of total downtime.

It impacts 20% of users. 300 minutes * 0.20 = 60 bad minutes per incident.

It happens 1.67 times a year. 60 * 1.67 = 100 bad minutes per year.

Why is this table so incredibly valuable for an SRE? Because it eliminates office politics. If you sort this spreadsheet by the highest 'bad mins/year', the system tells you exactly what to fix first.

Look at the data: a malicious security breach is costing us 570 bad minutes a year. A disk failure is costing us zero, likely because we already have redundant hardware. So, when the engineering team asks, 'What should we automate or improve this sprint?' we don't argue. We look at the table, tackle the 570-minute problem, and engineer a way to shrink its detection time, repair time, or frequency.

Risk Estimation

- Organization defines a Error Budget, which will affect the priorities of SRE staff
- If it exceeds the expected EB, it becomes high priority

	1242			
Threshold of unacceptability for an individual risk (% of error budget)	25.00%			
Target Availability	99.95%	99.90%	99.50%	
Budget (m/yr)	262.98	525.96	2629.8	
Accepted (m/yr)	0	0	0	
Unallocated Budget (m/yr)	262.98	525.96	2629.8	
Too Big Threshold (m/yr)	65.745	131.49	657.45	
Computed Stack Rank of Risks	bad mins/year	accept risk?	accept risk?	accept risk?
Malicious security breach/attack	570			
Third-party dependencies failure	152			
Aggressive database query causing system to down	110			
Software defects in the code	100			
Breakdown of caching service	81			
Disaster recovery drill	65			
Breakdown in the message queue system	51			
Code deployment misconfiguration	51			
Unintended data loss or corruption	28			
High volume of traffic	25			
Outage in the availability zone of the data center	7			
Update major code version	2			
Disk failure	0			

<https://dev.to/coingecko/error-budgets-in-practice-a-data-driven-approach-to-risk-and-release-management-2m9b>

GIC

Alright, let's bring it all together. On the last slide, we calculated exactly how many 'bad minutes per year' each risk would cost us. But a number like 110 bad minutes doesn't mean anything in a vacuum. We have to compare it against our organization's Error Budget.

Take a look at the top section of this table. Here we are modeling three different potential Service Level Objectives: 99.95%, 99.90%, and 99.50%.

Notice how the math cascades. If we choose a strict 99.95% target, our total yearly budget is only about 263 minutes. But look at the row labeled 'Threshold of unacceptability.' We have set a rule that no single risk is allowed to consume more than 25% of our total budget. Why? Because if one type of failure eats your whole budget, you have zero room left for deploying new features or dealing with unexpected anomalies.

So, 25% of 263 minutes gives us a 'Too Big Threshold' of roughly 66 minutes.

Now, look at the bottom half of the spreadsheet. We took our sorted list of risks from the previous slide and compared them against that 66-minute threshold. The results are color-coded.

Everything in red exceeds our 25% limit. If we are aiming for 99.95% reliability, the math objectively tells us we cannot accept the risk of a malicious security breach, a third-party failure, database queries crashing the system, or software defects. They are too expensive. As the slide says, 'If it exceeds the expected EB, it becomes high priority.' The SRE team now knows exactly what they must engineer away this quarter.

But look at what happens if the business decides that 99.50% reliability is acceptable (the column on the far right). Our budget expands to over 2,600 minutes. Our 25% threshold jumps to 657 minutes. Suddenly, almost the entire board turns green.

This is why SRE is so powerful. It proves to the business that if they want strict reliability (99.95%), they must allocate engineering time to fix those red boxes. If they don't want to spend the money fixing those red boxes, they must accept a lower SLO (99.50%). You cannot have it both ways.

Culture: Blameless Post-Mortems

- **Systems will fail:** Pods will OOMKill (Out of Memory). Network policies will misconfigure.

The Blameless Philosophy

- When the app goes down, we don't fire the engineer who pushed the bad YAML.
 - We ask: "Why did our CI/CD pipeline allow bad YAML to be applied?" or "Why didn't our K8s test environment catch this before production?"
-
- **Actionable Takeaways:** Every outage must result in an action item
 - writing a new automated test, adding a resource limit to a K8s manifest.

GIC

We've spent a lot of time looking at the math and the metrics of reliability. But now, we need to talk about what happens in the room after the outage is over. This brings us to one of the defining cultural pillars of DevOps and SRE: the Blameless Post-Mortem.

First, let's acknowledge reality: Systems will fail. No matter how good your risk matrix is, a Kubernetes Pod is eventually going to hit an OOMKill—an Out of Memory error. A developer is going to misconfigure a network policy. It is inevitable.

So, what happens when a junior engineer accidentally pushes a bad YAML file and takes down the production cluster for 20 minutes? In a traditional, toxic IT culture, the manager yells at the engineer, or worse, fires them.

But let's look at The Blameless Philosophy. In SRE, we assume good intent. We assume the engineer was trying to do their job, not sabotage the company. If we fire the engineer, what have we accomplished? We just fired the one person who now intimately understands exactly how the system breaks! More importantly, if you punish mistakes, engineers will start hiding their mistakes. When people hide mistakes, your Time to Detect (MTTD) skyrockets.

Instead of asking 'Who did this?', a blameless culture asks 'Why did the system allow this?' Why did our CI/CD pipeline accept a malformed YAML file? Why didn't our staging environment catch it? We don't fix the human; we fix the machine.

However, 'blameless' does not mean 'unaccountable.' That leads us to the final point: Actionable Takeaways. A post-mortem is completely useless if it just ends with people saying, 'Oops, we'll try harder next time.' Every single post-mortem must generate concrete Jira tickets. If the outage was caused by a memory leak, the action item is writing an automated test or adding strict resource limits to the Kubernetes manifest.

The goal of a blameless post-mortem isn't to make everyone feel warm and fuzzy; it's to ensure that we never, ever have the exact same outage twice.

How Kubernetes Fits

- Kubernetes is highly structured, and managed with YAML Files
 - Can be stored in code repository
 - Can be reviewed, audited and changed, with history kept
- It natively supports mechanisms and checks enabling SRE:
 - **Self-Healing:** If a node dies, K8s spins up a new pod. This eliminates the toil of manually restarting servers at 3 AM.
 - **Health Checks:** Liveness and Readiness probes are essentially automated SRE responders, ensuring traffic only goes to healthy pods.
 - Running pods != Healthy pods!
 - **Autoscaling (HPA):** HPA handles traffic spikes automatically, protecting the SLO without human intervention.
 - **Resource Definitions:** Each resource can validate their content (e.g. secrets), preventing errors.

GIC

We just talked about how a blameless culture requires us to fix the system rather than punish the human. But how do we actually build a system smart enough to fix itself? In modern DevOps and SRE, the answer is almost always Kubernetes.

Let's look at the first point on the slide. Kubernetes forces us to treat our operations like a software engineering problem. Instead of logging into a server and manually clicking through menus, everything in Kubernetes is highly structured and defined by YAML files.

Why is this a game-changer? Because it means our entire infrastructure can be stored in a code repository like Git. This is called Infrastructure as Code. When Dev and Ops share a Git repository, they share a common language. If an engineer wants to change a firewall rule or add a server, they submit a Pull Request. It gets reviewed, audited, and version-controlled. If it breaks production, rolling back is as easy as reverting a Git commit. This provides the exact audit trail we need for those blameless post-mortems we just discussed.

Now look at the bottom half of the slide. Kubernetes natively supports the mechanisms that enable SREs to sleep at night.

Self-Healing: Remember our definition of 'Toil'? Manually restarting crashed servers at 3:00 AM is the ultimate toil. Kubernetes watches your applications. If a pod crashes, Kubernetes instantly spins up a replacement. The human engineer sleeps, and the Mean Time to Repair (MTTR) drops to seconds.

Health Checks: This is a crucial concept. Just because a server is powered on does not mean it is healthy. An application can be deadlocked or frozen while still consuming CPU. Kubernetes uses Liveness and Readiness probes to constantly ask the app, 'Are you okay? Can you take traffic?' They act like automated, tireless SRE responders.

Autoscaling (HPA): We talked about protecting your Error Budgets and SLOs. If a massive traffic spike hits your website, Horizontal Pod Autoscaling (HPA) automatically detects the load and adds more servers to handle it, protecting your latency SLOs without requiring an engineer to manually provision hardware.

In short, Kubernetes is what happens when you take the SRE handbook and turn it into a software platform.